



FIT2096 - Games Programming

WEEK 03 LAB

Vectors and Rotations

Welcome to the Week 3! This lab is going to dive into the many ways we can use vectors and other math fundamentals to move and rotate actors in our game world.

We will create some basic moving platforms that will move between a number of points, and explore ways we can store and use position information in our lab.

We will explore actor rotations, and the limitations of using vector maths.

Our final project for this week will be an obstacle course for the player to navigate.

This Lab is Assessed

Each of the labs from weeks 1 - 9 are assessed as part of the Learning Portfolio. Please make sure to check Moodle and OnTrack for detailed information on each of the tasks and what needs to be submitted.

Learning outcomes for this week:

- Create a basic moving platform that travels between two points
- Create a platform that travels between a sequence of points
- Create a platform that rotates on the spot
- Create a platform that orbits a location
- Create a surveillance camera



The team is starting on a new prototype to show to a potential publisher, and we need you to help put it together.

Your task today is to create a new demo project for us with a cool obstacle course. We're hoping we can make the next Fall Guys, so we'll need lots of different types of platforms for traversal.

The game will be themed around spies and stealth missions, so we'd also like you to make a start on a surveillance camera that will follow the player around.

As always, good luck with your tasks!

- Project Lead

1. Create a New Git Repository

This week, we'll be starting with a **new Unreal Project**, which means we'll also need a **new git repository**. This process will be much the same as week 1, but we'll go over the steps again anyway.

We'll start by creating a new repository for our project.

Open the [Monash FIT GitLab](#) and click on groups from the left menu. Use the same group that contains your other lab tasks..

Select **New Project** from the top right of the group, then select **Create Blank Project**. Name it **P4_ObstacleCourse**.

Remember to **Untick** Initialize with Readme. We want a completely blank project. Now hit Create Project.

Create new project

**Create blank project**
Create a blank project to store your files, plan your work, and collaborate on code, among other things.

**Create from template**
Create a project pre-populated with the necessary files to get you started quickly.

**Import project**
Migrate your data from an external source like GitHub, Bitbucket, or another instance of GitLab.

**Run CI/CD for external repository**
Connect your external repository to GitLab CI/CD.

Visibility Level ?

☒ Private 
Project access must be granted explicitly to each user. If this project is part of a group, access is granted to members of the group.

☐ Internal 
The project can be accessed by any logged in user except external users.

☐ Public 
The project can be accessed without any authentication.

Project Configuration

☐ Initialize repository with a README
Allows you to immediately clone this project's repository. Skip this if you plan to push up an existing repository.

☐ Enable Static Application Security Testing (SAST)
Analyze your source code for known security vulnerabilities. [Learn more.](#)

☐ Enable Secret Detection
Scan your code for secrets and credentials to prevent unauthorized access. [Learn more.](#)

Create project Cancel

2. Create a New Unreal Project

Now you've got your repository set up, we can move onto creating our new project.

Launch Unreal Engine 5.7 and once the Project Browser pops up, select **Games** from the left side, and then select **Third Person**. With this selected, make sure you select the Project Defaults as follows:

Blueprint/C++: C++

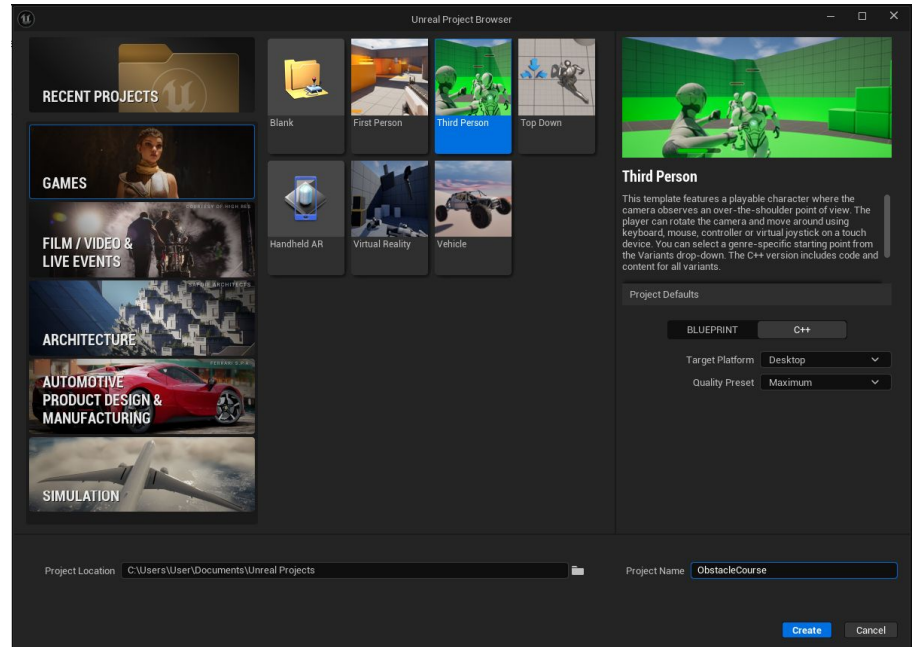
Target Platform: Desktop

Quality Preset: Scalable

Name your project

P4_ObstacleCourse and

select where you want to save your project.



Warning - Documents folder

Remember to save your work in the **local Documents folder** if you're on the lab machines, and ensure you **commit and push** your work at the end of the class, to ensure your work doesn't get wiped.

Click **Create** and wait for the project files to generate. This may take a while! Sit back and relax, or take the opportunity to meet some of your peers in your lab. If you're at home, maybe stretch and get a drink of water.

3. Commit Project to Git

At this point it is a good idea to commit and push your project using Git. The process is the same as in week 1. If you need to revise you can find the instructions at the following [link here](#) and under week 1 on Moodle

Git Problems?

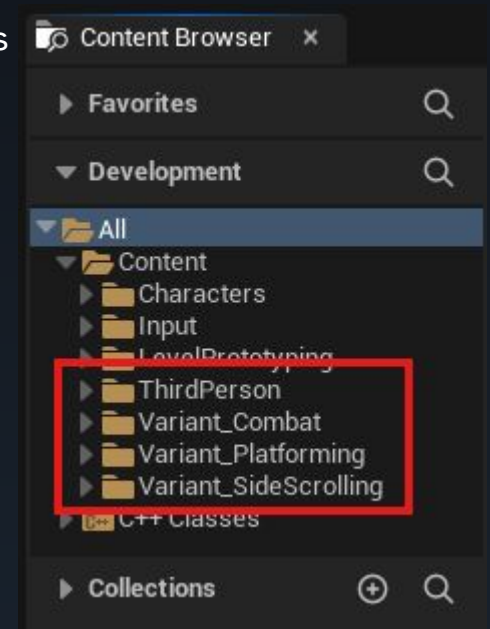
If you're running into problems with setting up your git repository **DO NOT SKIP SETTING THIS UP**. Go to the **Ed Forum** and look for the pinned comment showing multiple fixes for setting up your git repository. If none work please attend a consultation or ask your demonstrator for help.

Project templates in Unreal come with some basic classes to help you get started. Depending on what option you select, these will change.

We are using the Third Person project which comes with three variants of the basic player controller, each tuned for different gameplay.

Each folder contains some starting blueprints and other assets for each game mode. Try opening each folder, and check out the level contained in each.

Since we're using C++



4. Setting up our project workflow

Before we get started, there's a few things we can change to make our lives easier. These are optional, but will make your life easier.

Change your default source code editor to Rider

Go to **Edit > Editor Preferences > Source Code** and change the default editor to Rider Uproject. Hit **Set as default** and restart Unreal. This will stop Visual Studio from opening.

Swap the Startup Map

For this week's lab, we're going to use the Platforming variant of the template, as it will be a little better suited to our new features. You can swap the default map so that the editor opens it automatically each time you work on it.

In **Edit > Project Settings > Maps & Modes > Default Maps**, swap the **Editor Startup Map** and **Game Default Map** to **Lvl_Platforming**.

5. Create a basic platform

Now we can start building our obstacle course. Our first platform will be a simple platform that continuously moves between two points in a straight line.

In the C++ Classes folder, **right click** and **create a new c++ class**. Call this class **HorizontalPlatform** and select the parent class **Actor**. Click create and open Rider.

This platform will move two places back & forth. The first will be its spawn location, and the second will be a set amount of space in front of the platform. This "infront" will be forward on the World X-Axis.

To achieve this functionality we need the following variables in the header.

```
UPROPERTY(EditAnywhere, BlueprintReadWrite)
float MovementSpeed;
```

```
UPROPERTY(EditAnywhere, BlueprintReadWrite)
float MovementDistance;
```

These variables should be self explanatory. We can decide how fast the platform moves, and how far it should travel before returning.

We will also need some variables to keep track of where we started, and where we're trying to go.

```
UPROPERTY()
FVector PreviousStartLocation;
UPROPERTY()
FVector CurrentGoalLocation;
```

5. Create a basic platform (contd.)

We also need a variable for the tolerance threshold - this will help us “snap” to the final point, and help resolve issues with rounding errors.

```
UPROPERTY(EditAnywhere, BlueprintReadWrite)
float ToleranceThreshold;
```

Finally, we need a Mesh, as this is a platform we hope to see and interact with in the world.

```
UPROPERTY(EditAnywhere)
UStaticMeshComponent* PlatformMesh;
```

We don't need any special functions, so we can move straight to the CPP file and set up our functionality.

Constructor

The first step in the constructor is to set up sensible defaults for movement distance, speed and tolerance. We have exposed this to the editor so can change them in sub-classes as we see fit.

```
MovementSpeed = 100;
MovementDistance = 1000;
ToleranceThreshold = MovementSpeed/10;
```

The next step is to set up the defaults for both the Scene Component and Mesh.

```
RootComponent = CreateDefaultSubobject<USceneComponent>
    (TEXT("Root Component"));
PlatformMesh = CreateDefaultSubobject<UStaticMeshComponent>
    (TEXT("Platform Mesh"));
PlatformMesh->SetupAttachment(RootComponent);
```

BeginPlay

Inside the BeginPlay function we need to setup the location variables. This is done in BeginPlay and not the constructor to ensure that we already have a position in the world

```
PreviousStartLocation = GetActorLocation();
CurrentGoalLocation = GetActorLocation() +
    FVector(MovementDistance, 0, 0);
```

Note that we have hard-coded the distance to be on the X-Axis. This is the world forward direction and not the object.

6. Create a basic platform (contd.)

Tick

This is where the movement code will occur. Firstly we need to get the direction we want to move in. This is done with vector subtraction. **Remember: Destination Minus Origin**

```
FVector Direction = CurrentGoalLocation - GetActorLocation();  
Direction.Normalize();
```

As we only want the direction and not the distance we normalise the result to be a unit vector. We then multiply by our desired speed and delta time.

```
Direction *= MovementSpeed * DeltaTime;  
FVector NewLocation = GetActorLocation() + Direction;
```

As we only want the direction and not the distance we normalise the result to be a unit vector. We then multiply by our desired speed and delta time.

Next we need to check if we are “close enough” to our goal location. If we are then we can snap to it, and reset our goal / starting location.

```
if (FVector::Distance(CurrentGoalLocation, NewLocation) ≤  
ToleranceThreshold)  
{  
    NewLocation = CurrentGoalLocation;  
    CurrentGoalLocation = PreviousStartLocation;  
    PreviousStartLocation = NewLocation;  
}  
SetActorLocation(NewLocation);
```

With this the C++ version of this platform is done. **Save, compile** and **return to Unreal Engine**.

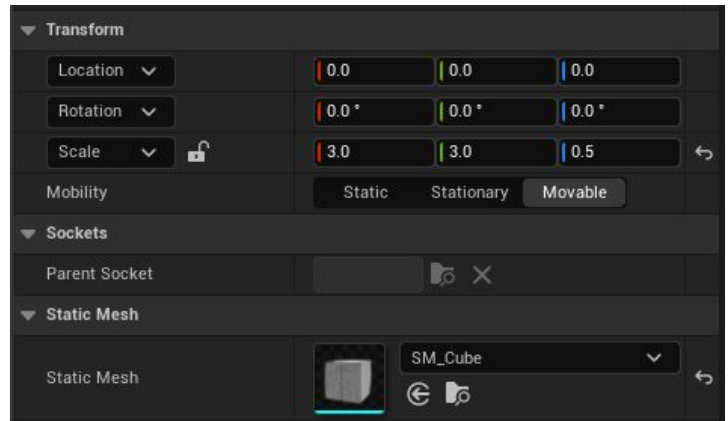
6. Create a basic platform (contd.)

With the base class done we can now create a blueprint subclass for use in the game world. **Right click** on the C++ class in Unreal Engine and **create a blueprint subclass**. Remember to use the prefix **BP_** in front of the class name and save it in the Blueprints folder.

Open the new subclass and select the Platform Mesh from the Components on the left. Then inside the details panel on the right locate **Static Mesh** and set it to **SM_Cube**.

Change the scale to **3.0, 3.0, 0.5** (or whatever suits you).

Save and **compile**.



Then drag a platform out from the Content Browser onto the game world. You should now be able to see the platform moving on its own.

7. Create a more flexible platform

Now, we have a basic moving platform, but it's pretty limited. Currently it can only move along one axis between two locations. We want a platform that can be given multiple locations to move to, and is not limited to a single direction.

Create a **new C++ class** and call it **WaypointPlatform**.

Class Variables:

There are a number of variables that will be needed for this platform, some of which are shared with the previous platform and some new ones. Let's start with the ones you will already be familiar with.

```
UPROPERTY(EditAnywhere)
UStaticMeshComponent* PlatformMesh;

UPROPERTY(EditAnywhere, BlueprintReadWrite)
float MovementSpeed;
UPROPERTY(EditAnywhere, BlueprintReadWrite)
float ToleranceThreshold;
```

7. Create a more flexible platform (contd.)

Next, instead of having a starting point and an end point as separate variables, we will set this class up to have n number of waypoints to move between. For this we will use a TArray of FVector locations.

```
UPROPERTY(EditAnywhere, BlueprintReadWrite)  
TArray<FVector> Waypoints;
```

We will set these waypoints in the editor later. Finally we need a variable to indicate the current index of the array we are up to.

```
int32 GoalIndex;
```

Once again, we won't need any additional functions, so we can **save** and go to the CPP file.

Constructor

Firstly, inside the constructor give movement speed, tolerance and goal index sensible defaults.

```
MovementSpeed = 100;  
ToleranceThreshold = MovementSpeed/10;  
GoalIndex = 0;
```

Next create the default objects for RootComponent and Mesh, and attach them.

```
RootComponent = CreateDefaultSubobject<USceneComponent>  
    (TEXT("Root Component"));  
PlatformMesh = CreateDefaultSubobject<UStaticMeshComponent>  
    (TEXT("Platform Mesh"));  
PlatformMesh->SetupAttachment(RootComponent);
```

Tick

Tick is where most of the functionality for this class will occur. Before any functionality occurs however, we need to ensure that we have waypoints to move towards AND our current index is valid. Otherwise the game would crash.

```
if (Waypoints.Num() == 0 || GoalIndex ≥ Waypoints.Num())  
    return;
```

7. Create a more flexible platform (contd.)

Once we know accessing the waypoints is safe, we can get the direction or movement using vector subtraction. Again **destination minus origin** gives us a vector pointing towards our goal.

```
FVector Direction = Waypoints[GoalIndex] - GetActorLocation();
Direction.Normalize();
Direction *= MovementSpeed * DeltaTime;
FVector NewLocation = GetActorLocation() + Direction;
```

Finally if we are “close enough” to our destination, snap to it and increment the goal index as we will want to move towards the next goal.

```
if (FVector::Distance(Waypoints[GoalIndex], NewLocation) ≤
ToleranceThreshold)
{
    NewLocation = Waypoints[GoalIndex];
    GoalIndex++;
}
SetActorLocation(NewLocation);
```

Save, compile and return to **Unreal**.

Just like before, we now need to create our blueprint subclass.

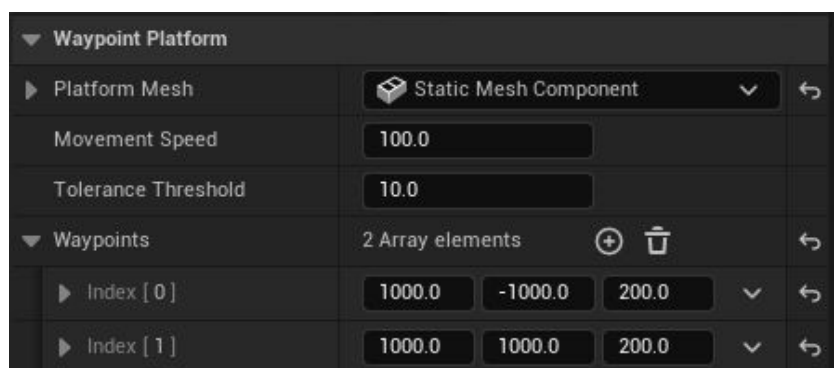
Right click on the C++ class in Unreal Engine and **create a blueprint subclass**. Use the prefix **BP_** in front of the class name and save it in the Blueprints folder.

Open the new subclass and select the Platform Mesh from the Components on the left. Then inside the details panel on the right locate **Static Mesh** and set it to **SM_Cube**. Change the scale to **3.0, 3.0, 0.5**.

Save and **compile**. Then drag a platform out from the Content Browser onto the game world.

Once the platform is within the game world, select it and open the **Waypoints vector** in the **details window**. Here we can manually add waypoints in the world. To travel between. **Add at least two with values** that make sense for your world location. If you have trouble select other objects and see their position in the world to help inform your decision.

You can now play the game and see your new platform working.



8. Creating a rotating platform

Next up is our rotating platform. This one is going to spin in place.

As before, we'll **create a new C++ class**, set the parent as Actor, and name it **RotatingPlatform**.

Class Variables:

This platform will have some slightly different variables, but they serve the same purpose. As always, we'll want a Mesh for our platform. We'll also want a variable to control how fast we rotate, and a variable to track the current rotation of the platform.

```
UPROPERTY(EditAnywhere)
UStaticMeshComponent* PlatformMesh;

UPROPERTY(EditAnywhere, BlueprintReadWrite)
float RotationSpeed;
UPROPERTY()
float CurrentRotation;
```

Constructor

We'll set up our Mesh and attach it to our Root Component, and set some values for our variables.

```
RootComponent = CreateDefaultSubobject<USceneComponent>
    (TEXT("Root Component"));
PlatformMesh = CreateDefaultSubobject<UStaticMeshComponent>
    (TEXT("Platform Mesh"));
PlatformMesh->SetupAttachment(RootComponent);

RotationSpeed = 200.0f;
CurrentRotation = 0.0f;
```

Tick

For our rotation, we'll update our current rotation based on the rotation speed and delta time. Because we're working in degrees, every full rotation (360deg) we'll subtract 360, putting us back at 0.

```
CurrentRotation += RotationSpeed * DeltaTime;
if (CurrentRotation ≥ 360)
{
    CurrentRotation -= 360;
}
```

8. Creating a rotating platform (contd.)

Next we create an `FRotator` that uses our current rotation to spin around the **Yaw**, which is the up axis. This spins it in place. Finally, we update our Actor Rotation.

```
FRotator Rotation = FRotator(0, CurrentRotation, 0);  
SetActorRotation(Rotation);
```

Save, compile and return to **Unreal**.

You should know the drill by now. **Create a Blueprint subclass**, name and save it according to conventions, then open it and set up the Static Mesh. Use the same values as the other platforms.

Compile and **save** and you can now put your rotating platform in the level.

9. Creating an orbiting platform

Next up is to create a platform that will orbit around a designated location. As before, we'll **create a new C++ class**, set the parent as Actor, and name it **OrbitingPlatform**.

Class Variables:

This should be familiar. We'll need a Mesh, and some variables to control the speed and distance at which we orbit our location. As before, we'll also need to track what our current rotation is.

```
UPROPERTY(EditAnywhere)
UStaticMeshComponent* PlatformMesh;

UPROPERTY(EditAnywhere, BlueprintReadWrite)
float OrbitSpeed;
UPROPERTY(EditAnywhere, BlueprintReadWrite)
float OrbitDistance;
UPROPERTY(EditAnywhere, BlueprintReadWrite)
float CurrentOrbitAngle;
```

Constructor

We'll set up our Mesh and attach it to our Root Component, and set some values for our variables.

```
RootComponent = CreateDefaultSubobject<USceneComponent>
    (TEXT("Root Component"));
PlatformMesh = CreateDefaultSubobject<UStaticMeshComponent>
    (TEXT("Platform Mesh"));
PlatformMesh->SetupAttachment(RootComponent);

OrbitSpeed = 90.0f;
OrbitDistance = 4.0f;
CurrentOrbitAngle = 0.0f;
```

9. Creating an orbiting platform (contd.)

Tick

To calculate our new position, we'll need to work out what angle we should be at from our starting position based on our speed and current angle.

First, we'll update our current orbit angle by adding our speed times delta time. We'll then convert this to radians, as we'll need them for the next operation.

Now, we can calculate what our X and Y locations should be by taking the cosine and sine of our new position respectively. Finally, we'll update our location, keeping our Z value the same as the starting position, so our platform remains on the level we placed it.

```
CurrentOrbitAngle += DeltaTime * OrbitSpeed;
if (CurrentOrbitAngle ≥ 360)
{
    CurrentOrbitAngle -= 360;
};

float Radians = FMath::DegreesToRadians(CurrentOrbitAngle);

float x = GetActorLocation().X + OrbitDistance *
    FMath::Cos(Radians);
float y = GetActorLocation().Y + OrbitDistance *
    FMath::Sin(Radians);

SetActorLocation(FVector(x, y, GetActorLocation().Z));
```

Save, compile and return to **Unreal**.

You should know the drill by now. **Create a Blueprint subclass**, name and save it according to conventions, then open it and set up the Static Mesh. Use the same values as the other platforms.

Compile and **save** and you can now put your platform that orbits around its spawn location.

10. Create a security camera

Next up, we're going to create a security "camera" that will watch the player wherever they go. This won't be a real camera, just a visual representation of one.

Start by **creating a new c++ class** and naming it **SecurityCamera**. Use Actor as the parent class.

Make sure to include the Character class inside the header file

We don't need much for the Header file - only a Mesh and a Target. We'll use a generic ACharacter* reference instead of our P4_ObstacleCourseCharacter, as this will mean we can use either the Platforming variant of the player character, or any other variant we want.

```
UPROPERTY(EditAnywhere)
UStaticMeshComponent* CameraMesh;

UPROPERTY(EditAnywhere, BlueprintReadWrite)
ACharacter* TargetCharacter;
```

We have no additional functions, so head straight over to the CPP file.

Constructor

As with all of our actors so far, we'll start by setting up a Mesh and attaching it to a new Root component.

```
RootComponent = CreateDefaultSubobject<USceneComponent>
    (TEXT("Root Component"));
CameraMesh = CreateDefaultSubobject<UStaticMeshComponent>
    (TEXT("Camera Mesh"));
CameraMesh->SetupAttachment(RootComponent);
```

BeginPlay

We'll need a reference to our target character. We can get this by calling GetFirstPlayerController(), as this will reference our player. We'll get the character component from this controller, and save that as our Target character.

```
TargetCharacter =
Cast<ACharacter>(GetWorld()->GetFirstPlayerController()->
    GetCharacter());
```

Tick

Now, for our functionality we can use a handy function in the UKismetMathLibrary called FindLookAtRotation(). This will calculate the rotation needed to look between the Start and End locations provided.

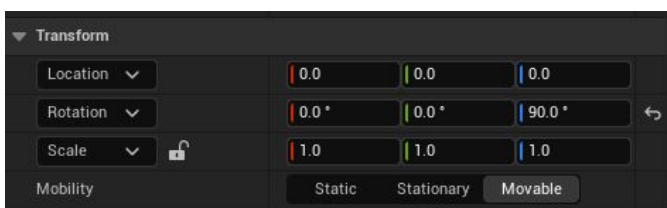
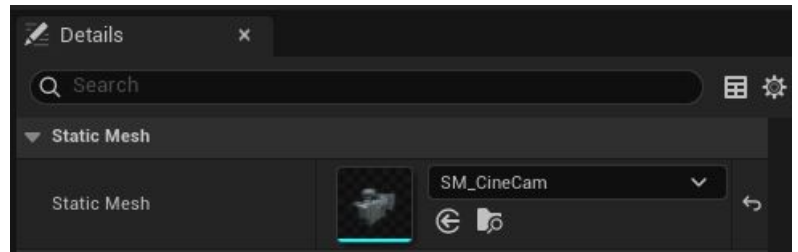
```
FQuat QuatRotation =
FQuat(UKismetMathLibrary::FindLookAtRotation(GetActorLocation(),
    TargetCharacter->GetActorLocation()));
SetActorRotation(QuatRotation);
```

10. Create a security camera (contd.)

Save, compile and return to **Unreal**.

Create a Blueprint subclass, name and save it according to conventions, then open it and set up the Static Mesh. You can use the SM_CineCam to inject a bit of realism. This also means we can see what direction the camera is facing, as opposed to a basic cube.

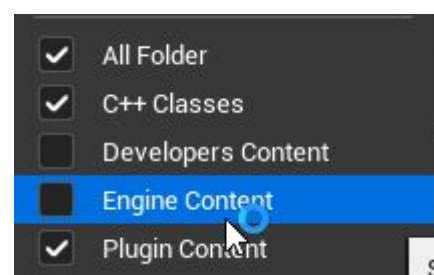
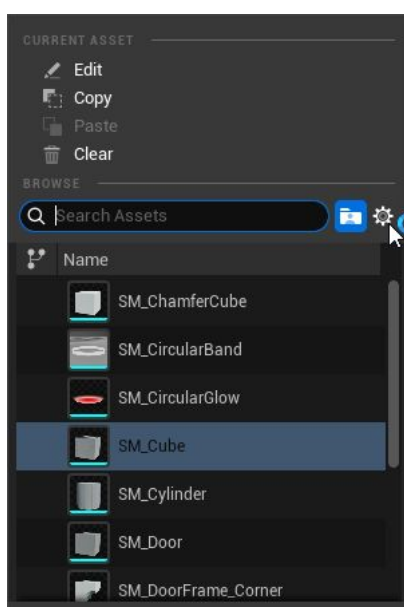
The camera Mesh faces a different direction to our Root component, so go to the Transform and **rotate the Mesh** by **90** degrees on the Z axis.



Compile and **save**, and place one in the scene. When you hit Play, the Camera should rotate to follow the player, no matter where they go.

Not Seeing SM_CineCam

SM_CineCam is a static mesh available in the engine. Sometimes depending on settings this may be hidden. In the drop down selection you can click the cog icon and tick Engine Content



11. Git Commit

You should now have a range of different waypoints that you can use to set up a fun little obstacle course. Run around and test your new features, and then **save** and **close** Unreal.

Be sure to **commit your new changes** in **Github Desktop**. As always, give it a meaningful commit name and description. Try and describe everything you did today in a few dot points so you can easily sort through your code in the future.

Once you've committed your work, be sure to **hit the Push button** otherwise the changes won't be sent to the remote repository!

Self Directed Tasks

Task 1 - Create a moving platform (looping)

The first self directed task is to create a platform that moves between four or more points in a sequence indefinitely (i.e continues to loop).

The task will require the following:

- A variable to store all of the points to move between
- A way to keep movement consistent between each point at a set speed

The waypoints must **NOT be hardcoded within C++** and should be done using a **UPROPERTY** that can be edited from within the Unreal Engine editor.

Task 2 - Create a moving platform (reversing)

The second self directed task is to create a platform that moves between four or more points, and reverses direction when it reaches the final waypoint. It should do this continuously.

The task will require the following:

- A variable to store all of the points to move between
- A way to keep movement consistent between each point at a set speed

Task 3 - Create platform that orbits a moving point

This task requires you to create a platform that orbits a point, **and follows it when it moves**.

This task will require the following:

- The orbiting platform must maintain its distance and rotation around the central object even if it moves